

Self-Supervised Learning from Images with a Joint- Embedding Predictive Architecture (I-JEPA)

CVPR-2023

Department of Computer Engineering and Artificial Intelligence
MAI Lab.

YS Park

POSTER

Self-Supervised Learning From Images With a Joint-Embedding Predictive Architecture

Mahmoud Assran · Quentin Duval · Ishan Misra · Piotr Bojanowski · Pascal Vincent · Michael Rabbat · Yann LeCun · Nicolas Ballas

2023 Poster

 Paper PDF

 Slides

 Poster

Abstract

This paper demonstrates an approach for learning highly semantic image representations without relying on hand-crafted data-augmentations. We introduce the Image-based Joint-Embedding Predictive Architecture (I-JEPA), a non-generative approach for self-supervised learning from images. The idea behind I-JEPA is simple: from a single context block, predict the representations of various target blocks in the same image. A core design choice to guide I-JEPA towards producing semantic representations is the masking strategy; specifically, it is crucial to (a) sample target blocks with sufficiently large scale (semantic), and to (b) use a sufficiently informative (spatially distributed) context block. Empirically, when combined with Vision Transformers, we find I-JEPA to be highly scalable. For instance, we train a ViT-Huge/14 on ImageNet using 16 A100 GPUs in under 72 hours to achieve strong downstream performance across a wide range of tasks, from linear classification to object counting and depth prediction.

- 해당 논문은 Yann LeCun (FAIR / NYU)의 역작 JEPA를 실제로 구현한 최초의 논문.
 - V-JEPA / VL-JEPA / **Brain-JEPA** 등 다양한 논문의 기초가 됨.
- JEPA는 AI를 World Model로 만들기 위해 제안된 방식이며, "생성"이 아닌 "예측"을 하는 것에 집중.
- 추후 fMRI 관련 연구인 Brain-JEPA / BrainHarmonix 순으로 논문 리뷰 예정

- Introduction
- Method
- Experiments
- Conclusion

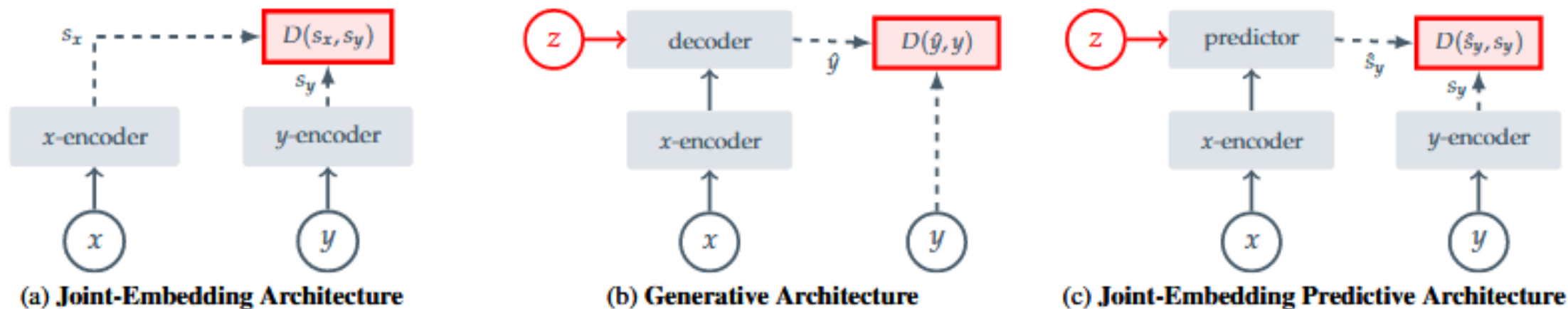
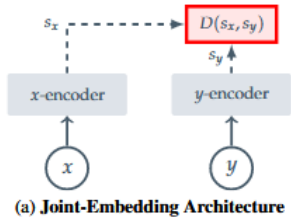
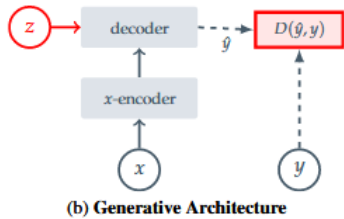


Figure 2. Common architectures for self-supervised learning, in which the system learns to capture the relationships between its inputs. The objective is to assign a high energy (large scalar value) to incompatible inputs, and to assign a low energy (low scalar value) to compatible inputs. (a) Joint-Embedding Architectures learn to output similar embeddings for compatible inputs x, y and dissimilar embeddings for incompatible inputs. (b) Generative Architectures learn to directly reconstruct a signal y from a compatible signal x , using a decoder network that is conditioned on additional (possibly latent) variables z to facilitate reconstruction. (c) Joint-Embedding Predictive Architectures learn to predict the embeddings of a signal y from a compatible signal x , using a predictor network that is conditioned on additional (possibly latent) variables z to facilitate prediction.



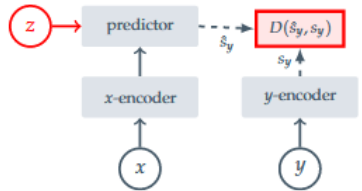
Joint-Embedding Architecture (JEA)

- Invariance-based Method
- Model: MoCo / SimCLR / VicReg / DINO / BYOL ... etc
- hand-crafted data augmentations에 의존함.
 - image-specific: random scaling, cropping, and color jittering ... etc
 - 다른 Modal에는 적용이 불가능함.
 - prior knowledge가 필요함.
- 강한 Inductive Bias / high semantic level을 제공함.
 - Low-level task나 다른 data distribution에 일반화가 어려움.
 - Representation collapse 막기 위한 다양한 서커스가 필요함.



Generative Architecture (GA)

- Reconstruction-based Method
- Model: MAE / CAE / data2vec / iBOT... etc
- JEA와 차이점
 - prior knowledge 덜 필요함.
 - 비교적 다양한 Modality에 일반화가 쉬움.
 - 약한 Inductive Bias / lower semantic level을 제공함.
 - Linear-probing X / Supervised Fine-Tuning (SFT)가 필요함.
 - Representation collapse가 보통 일어나지 않음.



(c) Joint-Embedding Predictive Architecture

Joint-Embedding Predictive Architecture

- Latent space에서 **prediction**
- JEA와 차이점
 - hand-crafted data augmentations를 사용하지 않음.
 - additional information z 에 따라 조건이 지정됨.
- GA와 차이점
 - Loss function이 embedding space에서 계산됨.

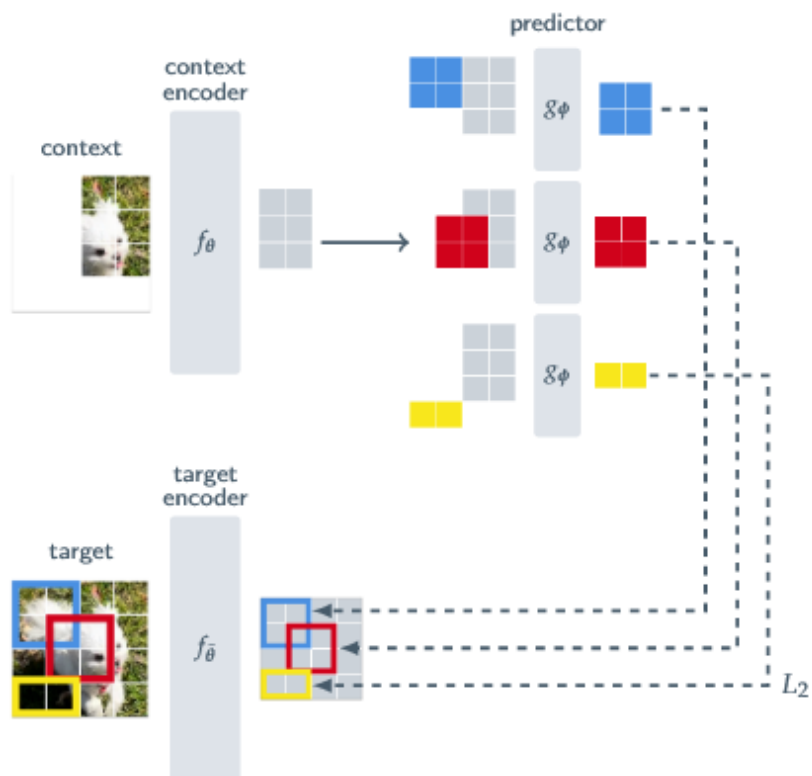


Figure 3. **I-JEPA**. The Image-based Joint-Embedding Predictive Architecture uses a single context block to predict the representations of various target blocks originating from the same image. The context encoder is a Vision Transformer (ViT), which only processes the visible context patches. The predictor is a narrow ViT that takes the context encoder output and, conditioned on positional tokens (shown in color), predicts the representations of a target block at a specific location. The target representations correspond to the outputs of the target-encoder, the weights of which are updated at each iteration via an exponential moving average of the context encoder weights.

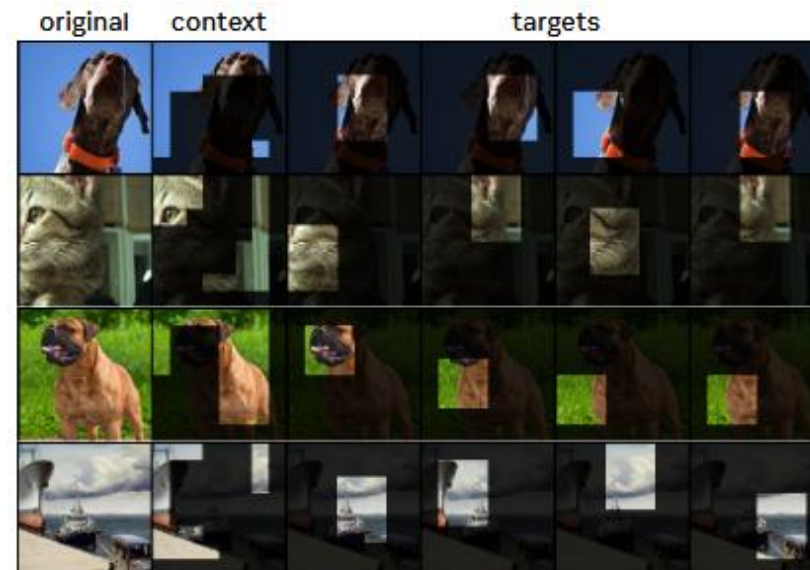
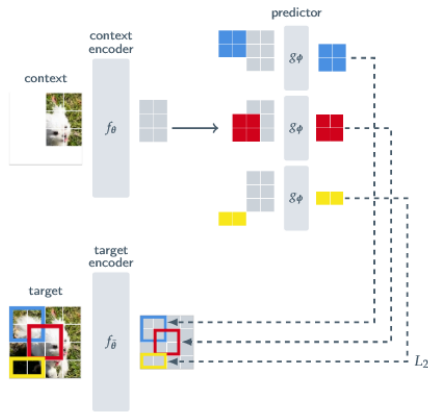


Figure 4. Examples of our context and target-masking strategy. Given an image, we randomly sample 4 target blocks with scale in the range (0.15, 0.2) and aspect ratio in the range (0.75, 1.5). Next, we randomly sample a context block with scale in the range (0.85, 1.0) and remove any overlapping target blocks. Under this strategy, the target-blocks are relatively semantic, and the context-block is informative, yet sparse (efficient to process).

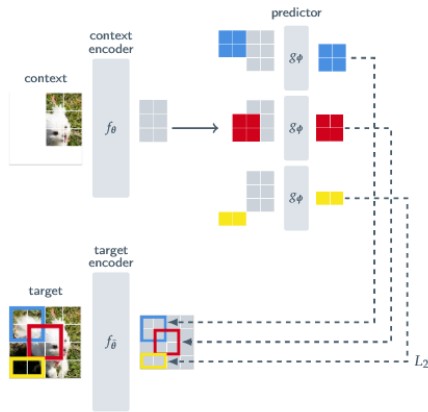


Goal: predict the **target** block representations from a single context block

Targets

- image $y \rightarrow N$ non-overlapping patches (보통 16×16)
- **Target encoder($f_{\bar{\theta}}$)**
 - patch-level representation $s_y = \{s_{y1}, \dots, s_{yN}\}$ 으로 인코딩
- 이 중에서 랜덤하게 M 개의 blocks을 sampling (이때 마스크끼리 overlap 가능)
 - $s_y(i) = \{s_{y_j}\}_{j \in B_i}$
 - 이때 B_i 는 i 번째 target block에 속하는 패치 인덱스들의 집합

파라미터	값	의미
M	4	target block 개수
aspect ratio	(0.75, 1.5)	직사각형
scale	(0.15, 0.2)	전체 이미지의 15~20% 크기



Goal: predict the target block representations from a single **context** block

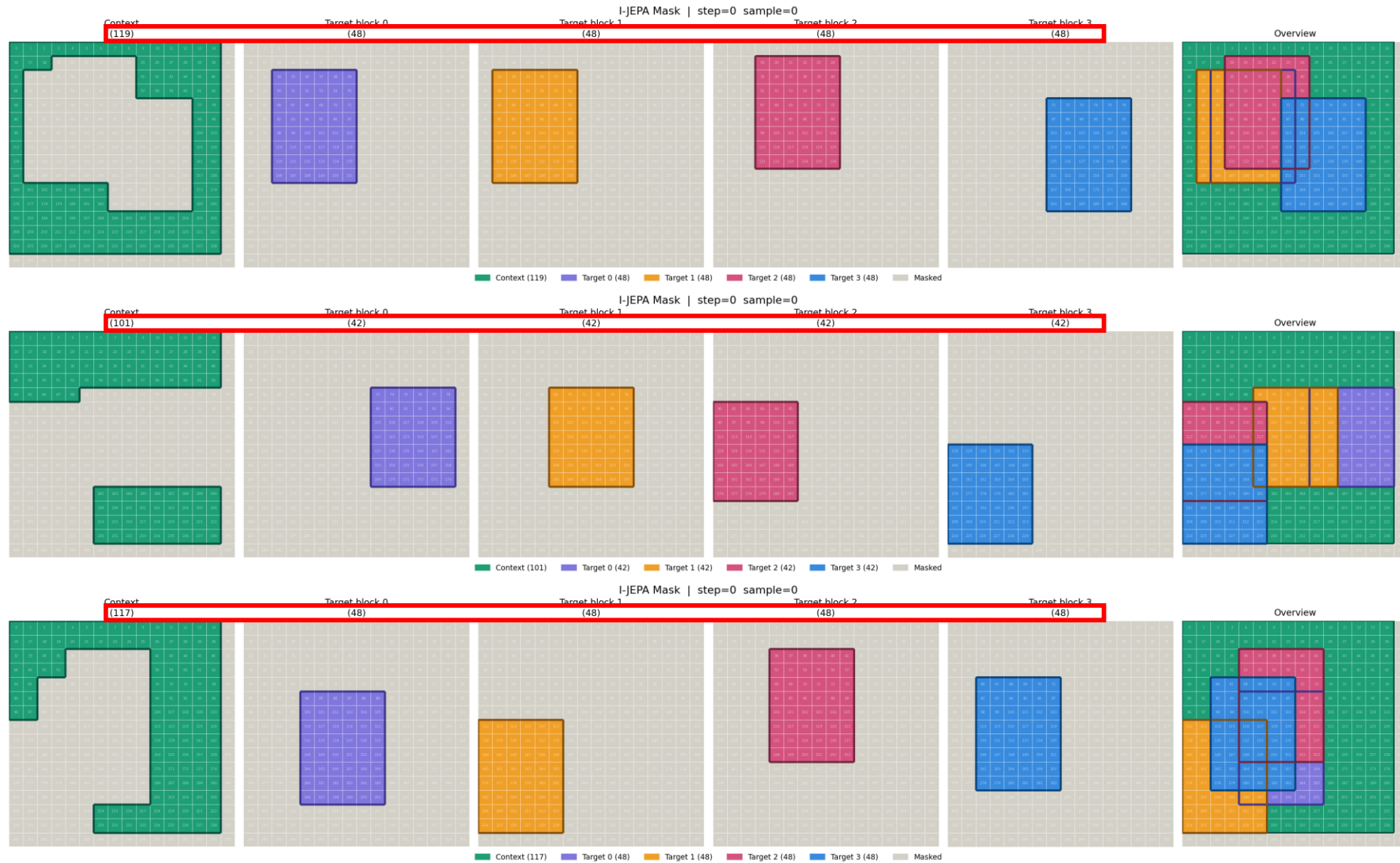
Context

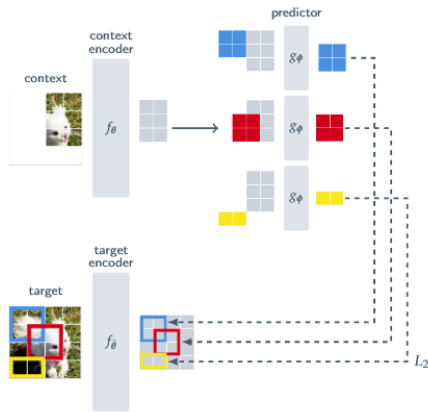
- image $y \rightarrow N$ non-overlapping patches (보통 16×16)
- **Context encoder (f_θ)**
 - patch-level representation $s_x = \{s_{x1}, \dots, s_{xN}\}$ 으로 인코딩
- block을 sampling (이때 치팅을 방지하기 위해 마스크에서 overlap되는 부분을 제거)
 - $s_x(i) = \{s_{xj}\}_{j \in B_x}$
 - 이때 B_x 는 context block에 속하는 패치 인덱스들의 집합

파라미터	값	의미
aspect ratio	(1, 1)	정사각형
scale	(0.85, 1.0)	전체 이미지의 85~100% 크기

○ ○ ○

```
def forward_target():  
    with torch.no_grad():  
        h = target_encoder(imgs)  
        # normalize over feature-dim  
        h = F.layer_norm(h, (h.size(-1),))  
        B = len(h)  
        # -- create targets (masked regions of h)  
        h = apply_masks(h, masks_pred)  
        h = repeat_interleave_batch(h, B, repeat=len(masks_enc))  
        return h  
  
def forward_context():  
    z = encoder(imgs, masks_enc)  
    z = predictor(z, masks_enc, masks_pred)  
    return z
```





Goal: **predict** the target block representations from a single context block

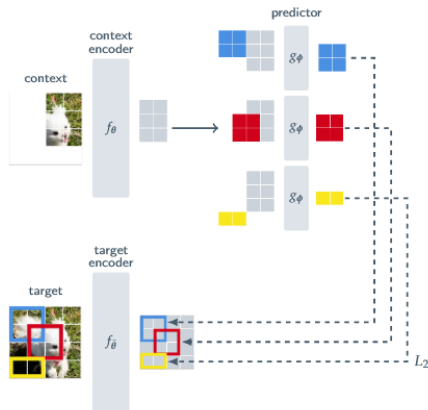
Prediction

- Predictor $g_\phi(\cdot, \cdot)$ predicts M target blocks representation $s_y(1), \dots, s_y(M)$ using context block s_x .
- $\hat{s}_y(i) = \{\hat{s}_{yj}\}_{j \in B_i} = g_\phi(s_x, \{m_j\}_{j \in B_i})$ where $\{m_j\}_{j \in B_i}$ is mask token.

○○○

```
def loss_fn(z, h):
    loss = F.smooth_l1_loss(z, h)
    loss = AllReduce.apply(loss)
    return loss
```

$$\neq \frac{1}{M} \sum_{i=1}^M D(\hat{s}_y(i), s_y(i)) = \frac{1}{M} \sum_{i=1}^M \sum_{j \in B_i} \|\hat{s}_{yj} - s_{yj}\|_2^2.$$



Goal: **predict** the target block representations from a single context block

Prediction

- Predictor $g_\phi(\cdot, \cdot)$ predicts M target blocks representation $s_y(1), \dots, s_y(M)$ using context block s_x .
- $\hat{s}_y(i) = \{\hat{s}_{yj}\}_{j \in B_i} = g_\phi(s_x, \{m_j\}_{j \in B_i})$ where $\{m_j\}_{j \in B_i}$ is mask token.
- Gradient-based optimization:
 - Predictor (g_ϕ) / Context encoder (f_θ)
- EMA-based optimization:
 - Target encoder ($f_{\bar{\theta}}$)

Published as a conference paper at ICLR 2026

DUAL PERSPECTIVES ON NON-CONTRASTIVE SELF-SUPERVISED LEARNING

Jean Ponce
 Ecole normale supérieure/PSL
 New York University
 jean.ponce@ens.fr

Basile Terver
 Meta FAIR
 INRIA Paris
 basileterv@meta.com

Martial Hebert
 Carnegie-Mellon University
 martial.hebert@cs.cmu.edu

Michael Arbel
 Univ. Grenoble Alpes, Inria
 CNRS, Grenoble INP, LJK
 michael.arbel@inria.fr

○○○

```
# Step 1. Forward
with torch.cuda.amp.autocast(dtype=torch.bfloat16, enabled=use_bfloat16):
    h = forward_target()
    z = forward_context()
    loss = loss_fn(z, h)

# Step 2. Backward & step
if use_bfloat16:
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
else:
    loss.backward()
    optimizer.step()
grad_stats = grad_logger(encoder.named_parameters())
optimizer.zero_grad()

# Step 3. momentum update of target encoder
with torch.no_grad():
    m = next(momentum_scheduler)
    for param_q, param_k in zip(encoder.parameters(), target_encoder.parameters()):
        param_k.data.mul_(m).add_((1.-m) * param_q.detach().data)

return (float(loss), _new_lr, _new_wd, grad_stats)
```

$$\theta_k \leftarrow m \cdot \theta_k + (1 - m) \cdot \theta_q$$

Architectures

- Context encoder (f_θ) & Target encoder ($f_{\bar{\theta}}$): standard ViT architectures (init. same weights)
- Predictor (g_ϕ):
 - Layer: 6 (ViT-B/16) or 12 or 16 (ViT-G/16)
 - fixed output dimension: 384D
 - [cls] X / average pool O

Optimization

- AdamW
 - weight-decay: 0.04 -> 0.4
- Learning Rate:
 - 10^{-4} -> 10^{-3} first 15 epochs
 - And decay to 10^{-6} following cosine schedule
- Momentum: 0.996 -> 1.0 (DINO)
- Batch size: 2048

Image Classification - ImageNet-1K & Low-Shot ImageNet-1K

Method	Arch.	Epochs	Top-1
<i>Methods without view data augmentations</i>			
data2vec [8]	ViT-L/16	1600	77.3
MAE [36]	ViT-B/16	1600	68.0
	ViT-L/16	1600	76.0
	ViT-H/14	1600	77.2
CAE [22]	ViT-B/16	1600	70.4
	ViT-L/16	1600	78.1
I-JEPA	ViT-B/16	600	72.9
	ViT-L/16	600	77.5
	ViT-H/14	300	79.3
	ViT-H/16 ₄₄₈	300	81.1
<i>Methods using extra view data augmentations</i>			
SimCLR v2 [21]	RN152 (2×)	800	79.1
DINO [18]	ViT-B/8	300	80.1
iBOT [79]	ViT-L/16	250	81.0

Table 1. **ImageNet**. Linear-evaluation on ImageNet-1k (the ViT-H/16₄₄₈ is pretrained at a resolution of 448×448). I-JEPA improves linear probing performance compared to other methods that do not rely on hand-crafted view data-augmentations during pre-training. Moreover, I-JEPA demonstrates good scalability — the larger I-JEPA model matches the performance of view-invariance approaches without requiring view data-augmentations.

Method	Arch.	Epochs	Top-1
<i>Methods without view data augmentations</i>			
data2vec [8]	ViT-L/16	1600	73.3
MAE [36]	ViT-L/16	1600	67.1
	ViT-H/14	1600	71.5
I-JEPA	ViT-L/16	600	69.4
	ViT-H/14	300	73.3
	ViT-H/16 ₄₄₈	300	77.3
<i>Methods using extra view data augmentations</i>			
iBOT [79]	ViT-B/16	400	69.7
DINO [18]	ViT-B/8	300	70.0
SimCLR v2 [35]	RN151 (2×)	800	70.2
BYOL [35]	RN200 (2×)	800	71.2
MSN [4]	ViT-B/4	300	75.7

Table 2. **ImageNet-1%**. Semi-supervised evaluation on ImageNet-1K using only 1% of the available labels. Models are adapted via fine-tuning or linear-probing, depending on whichever works best for each respective method. ViT-H/16₄₄₈ is pretrained at a resolution of 448×448 . I-JEPA pretraining outperforms MAE which also does not rely on hand-crafted data-augmentations during pretraining. Moreover, I-JEPA benefits from scale. A ViT-H/16 trained at resolution 448 surpasses previous methods including methods that leverage extra hand-crafted data-augmentations.

Image Classification - Transfer learning & Local Prediction Tasks

Method	Arch.	CIFAR100	Places205	iNat18
<i>Methods without view data augmentations</i>				
data2vec [8]	ViT-L/16	81.6	54.6	28.1
MAE [36]	ViT-H/14	77.3	55.0	32.9
I-JEPA	ViT-H/14	87.5	58.4	47.6
<i>Methods using extra view data augmentations</i>				
DINO [18]	ViT-B/8	84.9	57.9	55.9
iBOT [79]	ViT-L/16	88.3	60.4	57.3

Table 3. **Linear-probe transfer for image classification.** Linear-evaluation on downstream image classification tasks. I-JEPA significantly outperforms previous methods that also do not use augmentations (MAE and data2vec), and decreases the gap with the best view-invariance-based methods that leverage hand-crafted data augmentations during pretraining.

Method	Arch.	Clevr/Count	Clevr/Dist
<i>Methods without view data augmentations</i>			
data2vec [8]	ViT-L/16	85.3	71.3
MAE [36]	ViT-H/14	90.5	72.4
I-JEPA	ViT-H/14	86.7	72.4
<i>Methods using extra data augmentations</i>			
DINO [18]	ViT-B/8	86.6	53.4
iBOT [79]	ViT-L/16	85.7	62.8

Table 4. **Linear-probe transfer for low-level tasks.** Linear-evaluation on downstream low-level tasks consisting of object counting (Clevr/Count) and depth prediction (Clevr/Dist). The I-JEPA method effectively captures low-level image features during pretraining and outperforms view-invariance based methods on tasks such object counting and depth prediction.

Scalability - Model Efficiency & Scaling model size Predictor Visualizations with RCDM framework.

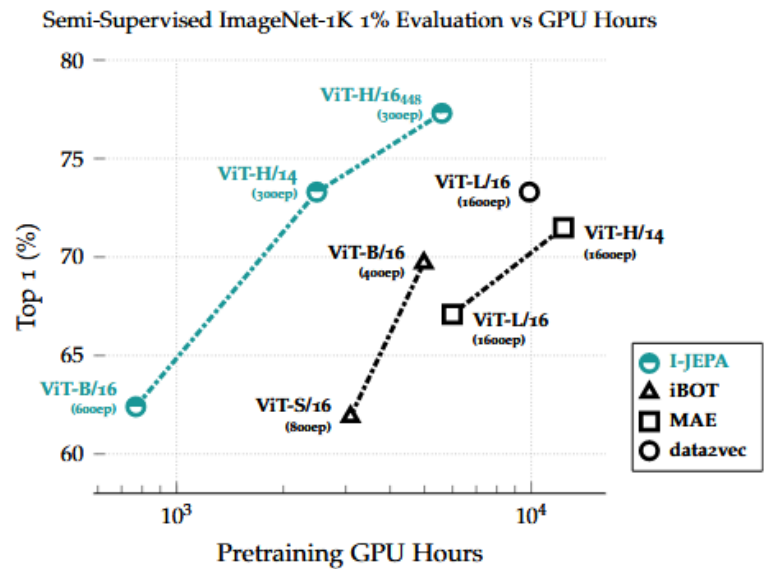


Figure 5. **Scaling.** Semi-supervised evaluation on ImageNet-1K 1% as a function of pretraining GPU hours. I-JEPA requires less compute than previous methods to achieve strong performance. Compared to MAE and data2vec, I-JEPA obtains a significant speedup by requiring fewer pretraining epochs. Compared to iBOT, which relies on hand-crafted data-augmentations, a huge I-JEPA model (ViT-H/14) requires less compute than their smallest model (ViT-S/16).

Pretrain	Arch.	CIFAR100	Place205	INat18	Clevr/Count	Clevr/Dist
IN1k	ViT-H/14	87.5	58.4	47.6	86.7	72.4
IN22k	ViT-H/14	89.5	57.8	50.5	88.6	75.0
IN22k	ViT-G/16	89.5	59.1	55.3	86.7	73.0

Table 5. **Ablating dataset and model size.** Evaluating impact of pre-training dataset size and model size on transfer tasks. I-JEPA benefits from larger more diverse datasets. When increasing the size of the pretraining dataset (IN1k versus IN22k) we see an performance improvement for the ViT-H/14 model. We observe a further performance improvement on semantic tasks by training a larger model ViT-G/16 model on ImageNet-22k. The ViT-H/14 is trained for 300 epochs on IN1k and the equivalent of 900 IN1K epochs on IN22k. The ViT-H/16 is trained for the equivalent of 600 IN1k epochs.

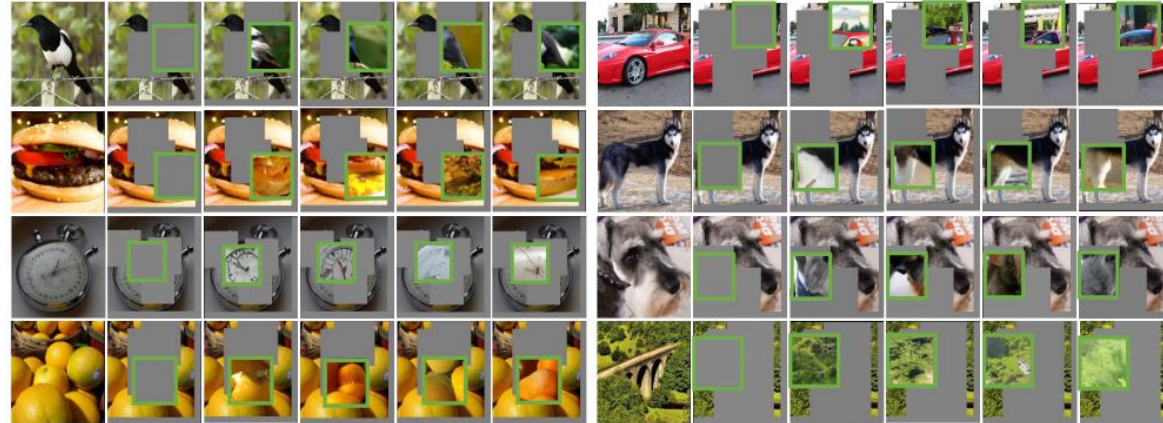


Figure 6. **Visualization of I-JEPA predictor representations.** For each image: first column contains the original image; second column contains the context image, which is processed by a pretrained I-JEPA ViT-H/14 encoder. Green bounding boxes in subsequent columns contain samples from a generative model decoding the output of the pretrained I-JEPA predictor, which is conditioned on positional mask tokens corresponding to the location of the green bounding box. Qualities that are common across samples represent information that contained is in the I-JEPA prediction. The I-JEPA predictor is correctly capturing positional uncertainty and producing high-level object parts with a correct pose (e.g., the back of the bird and top of a car). Qualities that vary across samples represent information that is not contained in the representation. In this case, the I-JEPA predictor discards the precise low-level details as well as background information.

Ablations

Targets	Arch.	Epochs	Top-1
Target-Encoder Output	ViT-L/16	500	66.9
Pixels	ViT-L/16	800	40.7

Table 7. **Ablating targets.** Linear evaluation on ImageNet-1K using only 1% of the available labels. The semantic level of the I-JEPA representations degrades significantly when the loss is applied in pixel space, rather than representation space, highlighting the importance of the target-encoder during pretraining.

Mask	Targets		Context		Top-1
	Type	Freq.	Type	Avg. Ratio*	
multi-block	Block(0.15, 0.2)	4	Block(0.85, 1.0) $\times$ Complement	0.25	54.2
rasterized	Quadrant	3	Complement	0.25	15.5
block	Block(0.6)	1	Complement	0.4	20.2
random	Random(0.6)	1	Complement	0.4	17.6

*Avg. Ratio is the average number of patches in the context block relative to the total number of patches in the image.

Table 6. **Ablating masking strategy.** Linear evaluation on ImageNet-1K using only 1% of the available labels after I-JEPA pretraining of a ViT-B/16 for 300 epochs. Comparison of proposed multi-block masking strategy. In rasterized masking the image is split into four large quadrants; one quadrant is used as a context to predict the other three quadrants. In block masking, the target is a single image block and the context is the image complement. In random masking, the target is a set of random image patches and the context is the image complement. The proposed multi-block masking strategy is helpful for guiding I-JEPA to learn semantic representations.

Ablations – Appendix C

Targets		Context	
Scale	Freq.	Scale	Top-1
(0.075, 0.2)	4	(0.85, 1.0)	19.2
(0.1, 0.2)	4	(0.85, 1.0)	39.2
(0.125, 0.2)	4	(0.85, 1.0)	42.4
(0.15, 0.2)	4	(0.85, 1.0)	54.2
(0.2, 0.25)	4	(0.85, 1.0)	38.9
(0.2, 0.3)	4	(0.85, 1.0)	33.6

Table 8. **Ablation of the target block size for multi-block masking.** Linear evaluation on 1% ImageNet-1K (using only 1% of the available labels); ablating the multi-block target size during I-JEPA pretraining of a ViT-B/16 for 300 epochs. Predicting larger (semantic) blocks improves the low-shot accuracy as long as the context is sufficiently informative.

Targets		Context	
Scale	Freq.	Scale	Top-1
(0.15, 0.2)	4	(0.40, 1.0)	31.2
(0.15, 0.2)	4	(0.65, 1.0)	47.1
(0.15, 0.2)	4	(0.75, 1.0)	49.3
(0.15, 0.2)	4	(0.85, 1.0)	54.2

Table 9. **Ablation of the context size for multi-block masking.** Linear evaluation on 1% ImageNet-1K (using only 1% of the available labels); ablating the multi-block target size during I-JEPA pretraining of a ViT-B/16 for 300 epochs. Reducing the multi-block context size degrades the low-shot performance.

Targets		Context	
Scale	Freq.	Scale	Top-1
(0.15, 0.2)	1	(0.85, 1.0)	9.0
(0.15, 0.2)	2	(0.85, 1.0)	22.0
(0.15, 0.2)	3	(0.85, 1.0)	48.5
(0.15, 0.2)	4	(0.85, 1.0)	54.2

Table 10. **Ablation of the targets number for multi-block masking.** Linear evaluation on 1% ImageNet-1K (using only 1% of the available labels); ablating the multi-block number of targets during I-JEPA pretraining of a ViT-B/16 for 300 epochs. Increasing the number of target blocks improve the low-shot accuracy.

Target Masking	Arch.	Epochs	Top-1
Output	ViT-H/16	300	67.3
Input	ViT-H/16	300	56.1

Table 11. **Ablating masking output of target encoder.** Linear evaluation on ImageNet-1K using only 1% of the available labels; ablating the effect of masking the target-encoder output during I-JEPA pretraining of a ViT-H/16 for 300 epochs. Masking the output of the target-encoder during pretraining significantly improves the linear probing performance of the pretrained representations.

Predictor depth. We examine the impact of the predictor depth on the downstream low-shot performance in Table 12. We pretrain a ViT-L/16 for 500 epochs using either a 6-layer predictor network or a 12-layer predictor network. The model pretrained using a deeper predictor shows a significant improvement in downstream low-shot performance compared to the model pretrained with a shallower predictor.

Predictor Depth	Arch.	Epochs	Top-1
6	ViT-L/16	500	64.0
12	ViT-L/16	500	66.9

Table 12. **Ablating the predictor depth.** Linear evaluation on ImageNet-1K using only 1% of the available labels; ablating the effect of masking the predictor depth for a ViT-L/16 pretrained for 500 epochs. Increasing the predictor depth leads to significant improvement of the linear probe performance of the pretrained representations.

Weight decay. In Table 13, we evaluate the impact of weight-decay during pretraining. We explore two weight decay strategies: linearly increase the weight-decay from 0.04 to 0.4 or use a fix weight-decay of 0.05. Using a smaller weight decay during pretraining improves the downstream performance on ImageNet-1% when fine-tuning. However, this also leads to a degradation of performance in linear evaluation. In the main paper, we use the first weight decay strategy as it improves the performances in linear evaluation downstream tasks.

Weight Decay	Arch.	Epochs	ImageNet-1%	ImageNet Linear-Eval
0.04 → 0.4	ViT-L/16	600	69.4	77.8
0.05	ViT-L/16	600	70.7	76.4

Table 13. **Ablating the pretraining weight-decay.** We compare our default pretraining weight decay strategy where we linearly increase the weight-decay from 0.04 to 0.4 to using a fix weight decay of 0.05. Using a smaller weight-decay during pretraining can improve the fine-tuning performance on ImageNet-1%. However, it also leads to a drop of performance in linear evaluation.

Predictor width. We explore the impact of the predictor width in Table 14. We compare I-JEPA using a ViT-L encoder and a predictor with 386 channels to a similar model using a predictor with 1024 channels. Note that the ViT-L encoder has 1024 channels. Using a bottleneck in the predictor width improves the downstream performance on ImageNet 1%.

Predictor Width	Arch.	Epochs	Top-1
384	ViT-L/16	600	70.7
1024	ViT-L/16	600	68.4

Table 14. **Ablating the predictor width.** We reports results on ImageNet-1K 1% using fine-tuning. We compare two predictors having a width of either 384 or 1024. Note the I-JEPA encoder is a ViT-L with 1024 channels. Having a width bottleneck in the predictor improves the downstream performances.

- We proposed I-JEPA, a **simple and efficient method** for learning **semantic image representations** without **relying on hand-crafted data augmentations**.
- We show that **by predicting in representation space**, I-JEPA **converges faster** than pixel reconstruction methods and **learns representations of a high semantic level**.
- In contrast to view-invariance based methods, I-JEPA highlights a path for **learning general representations** with joint-embedding architectures, without **relying on hand-crafted view augmentations**.